VIDHYADEEP UNIVERSITY
Holy Flame Of Knowledge

Vidhyadeep Institute of Commerce & Information Technology, Anita (Kim)

PRACTICAL RECORD

WEB FRAMEWORK AND SCRIPTING (PHP)

TY B.Sc. IT | Semester V

STUDENT NAME	SHIVAM BHADORIYA	ENROLLMENT NO.	240606302109

INDEX

NO.	PROGRAM / PRACTICAL	PAGE	SIGN
ASSIGNMENT 2 - PHP, MYSQL AND AJAX			
01	Employee Management System	2	
02	Course Management System	3	
03	Product Management and WHERE Search	4	
04	Customer Management with AJAX	5	
05	Mobile Shop Management with AJAX	6	

ASSIGNMENT 2 - PHP, MYSQL AND AJAX

Web Framework & Scripting

01. Employee Management System

Aim: Develop a PHP and MySQL website to insert, display, update and delete employee records, including ID, name, email, department, designation and salary.

Logic: MySQL stores the records; PDO implements parameterized CRUD; The records are displayed in the main table.

PROGRAM / model.php

```
<?php
return [
    'title' => 'Employee Management System',
    'database' => 'employee_db',
    'table' => 'employees',
    'pk' => 'id',
    'order' => 'id',
    'list_label' => 'Employee List',
    'ajax' => false,
    'search' => [],
    'fields' => [
        'name' => ['label' => 'Name', 'type' => 'text', 'max' => 100],
        'email' => ['label' => 'Email', 'type' => 'email', 'max' => 140],
        'department' => ['label' => 'Department', 'type' => 'text', 'max' => 80],
        'designation' => ['label' => 'Designation', 'type' => 'text', 'max' => 80],
        'salary' => ['label' => 'Salary', 'type' => 'number', 'max' => 30],
    ],
];
```

PROGRAM / RUN_ME.md

```
# Employee Management System

PHP 8.0+ and the **pdo_mysql** extension, MySQL 8.0+/MariaDB.

1. Import `schema.sql` using phpMyAdmin or the `mysql` command line.
2. Configure `DB_HOST`, `DB_USER`, `DB_PASSWORD` environment variables (or edit defaults in `db.php` for
local development).
3. From this program folder run `php -S 127.0.0.1:8000`.
4. Visit http://127.0.0.1:8000/index.php and test create/edit/delete.
5. Inspect the displayed records.

Do not deploy the default local root/blank-password database configuration to production.
This package has passed PHP syntax checks, not database-backed runtime tests.
```

PROGRAM / schema.sql

```
CREATE DATABASE IF NOT EXISTS `employee_db` CHARACTER SET utf8mb4;
USE `employee_db`;
CREATE TABLE IF NOT EXISTS `employees` (
  `id` INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  `name` VARCHAR(100) NOT NULL,
  `email` VARCHAR(140) NOT NULL,
  `department` VARCHAR(80) NOT NULL,
  `designation` VARCHAR(80) NOT NULL,
  `salary` DECIMAL(12,2) NOT NULL CHECK (`salary` >= 0)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

Common project files (reproduced in the appendix): common.php, db.php, delete.php, edit.php, index.php, rows.php, style.css.

EXPECTED OUTPUT

```
Insert -> new row appears in list.
Edit -> changes are persisted.
Delete -> record removed after confirmation.
Records appear in the list.
```

Expected interaction only; no MySQL server or pdo_mysql driver available on the execution host.

Note: Run schema.sql in MySQL, set DB_HOST/DB_USER/DB_PASSWORD, then serve with PHP 8.0+ and the pdo_mysql extension. Search and CRUD use prepared statements; output is escaped; mutations use POST and CSRF checks.

02. Course Management System

Aim: Implement PHP/MySQL Insert, Display, Update and Delete for course ID, name, duration, fees and description.

Logic: MySQL stores the records; PDO implements parameterized CRUD; The records are displayed in the main table.

PROGRAM / model.php

```
<?php
return [
    'title' => 'Course Management System',
    'database' => 'course_db',
    'table' => 'courses',
    'pk' => 'course_id',
    'order' => 'course_id',
    'list_label' => 'Course List',
    'ajax' => false,
    'search' => [],
    'fields' => [
        'course_name' => ['label' => 'Course Name', 'type' => 'text', 'max' => 120],
        'duration' => ['label' => 'Duration', 'type' => 'text', 'max' => 60],
        'fees' => ['label' => 'Fees', 'type' => 'number', 'max' => 30],
        'course_description' => ['label' => 'Course Description', 'type' => 'textarea', 'max' => 1000],
    ],
];
```

PROGRAM / RUN_ME.md

```
# Course Management System

PHP 8.0+ and the **pdo_mysql** extension, MySQL 8.0+/MariaDB.

1. Import `schema.sql` using phpMyAdmin or the `mysql` command line.
2. Configure `DB_HOST`, `DB_USER`, `DB_PASSWORD` environment variables (or edit defaults in `db.php` for
local development).
3. From this program folder run `php -S 127.0.0.1:8000`.
4. Visit http://127.0.0.1:8000/index.php and test create/edit/delete.
5. Inspect the displayed records.

Do not deploy the default local root/blank-password database configuration to production.
This package has passed PHP syntax checks, not database-backed runtime tests.
```

PROGRAM / schema.sql

```
CREATE DATABASE IF NOT EXISTS `course_db` CHARACTER SET utf8mb4;
USE `course_db`;
CREATE TABLE IF NOT EXISTS `courses` (
  `course_id` INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  `course_name` VARCHAR(120) NOT NULL,
  `duration` VARCHAR(60) NOT NULL,
  `fees` DECIMAL(12,2) NOT NULL CHECK (`fees` >= 0),
  `course_description` TEXT NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

Common project files (reproduced in the appendix): common.php, db.php, delete.php, edit.php, index.php, rows.php, style.css.

EXPECTED OUTPUT

```
Insert -> new row appears in list.
Edit -> changes are persisted.
Delete -> record removed after confirmation.
Records appear in the list.
```

Expected interaction only; no MySQL server or pdo_mysql driver available on the execution host.

Note: Run schema.sql in MySQL, set DB_HOST/DB_USER/DB_PASSWORD, then serve with PHP 8.0+ and the pdo_mysql extension. Search and CRUD use prepared statements; output is escaped; mutations use POST and CSRF checks.

03. Product Management and WHERE Search

Aim: Create the database/table, insert and display products, and search by product name or category using WHERE.

Logic: MySQL stores the records; PDO implements parameterized CRUD; A server-side WHERE query filters products.

PROGRAM / model.php

```
<?php
return [
    'title' => 'Product Management and WHERE Search',
    'database' => 'product_db',
    'table' => 'products',
    'pk' => 'product_id',
    'order' => 'product_id',
    'list_label' => 'Product List & Search',
    'ajax' => false,
    'search' => ['product_name', 'category'],
    'fields' => [
        'product_name' => ['label' => 'Product Name', 'type' => 'text', 'max' => 120],
        'category' => ['label' => 'Category', 'type' => 'text', 'max' => 80],
        'price' => ['label' => 'Price', 'type' => 'number', 'max' => 30],
        'stock' => ['label' => 'Stock', 'type' => 'integer', 'max' => 20],
    ],
];
```

PROGRAM / RUN_ME.md

```
# Product Management and WHERE Search

PHP 8.0+ and the **pdo_mysql** extension, MySQL 8.0+/MariaDB.

1. Import `schema.sql` using phpMyAdmin or the `mysql` command line.
2. Configure `DB_HOST`, `DB_USER`, `DB_PASSWORD` environment variables (or edit defaults in `db.php` for
local development).
3. From this program folder run `php -S 127.0.0.1:8000`.
4. Visit http://127.0.0.1:8000/index.php and test create/edit/delete.
5. Search using the form with a product name or category.

Do not deploy the default local root/blank-password database configuration to production.
This package has passed PHP syntax checks, not database-backed runtime tests.
```

PROGRAM / schema.sql

```
CREATE DATABASE IF NOT EXISTS `product_db` CHARACTER SET utf8mb4;
USE `product_db`;
CREATE TABLE IF NOT EXISTS `products` (
  `product_id` INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  `product_name` VARCHAR(120) NOT NULL,
  `category` VARCHAR(80) NOT NULL,
  `price` DECIMAL(12,2) NOT NULL CHECK (`price` >= 0),
  `stock` INT UNSIGNED NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

Common project files (reproduced in the appendix): common.php, db.php, delete.php, edit.php, index.php, rows.php, style.css.

EXPECTED OUTPUT

```
Insert -> new row appears in list.
Edit -> changes are persisted.
Delete -> record removed after confirmation.
Searching -> matched product rows displayed.
```

Expected interaction only; no MySQL server or pdo_mysql driver available on the execution host.

Note: Run schema.sql in MySQL, set DB_HOST/DB_USER/DB_PASSWORD, then serve with PHP 8.0+ and the pdo_mysql extension. Search and CRUD use prepared statements; output is escaped; mutations use POST and CSRF checks.

04. Customer Management with AJAX

Aim: Implement customer CRUD, AJAX live search by name or city using WHERE and alphabetical ordering by name.

Logic: MySQL stores the records; PDO implements parameterized CRUD; JavaScript fetch updates search results without reloading the page.

PROGRAM / model.php

```
<?php
return [
    'title' => 'Customer Management with AJAX',
    'database' => 'customer_db',
    'table' => 'customers',
    'pk' => 'customer_id',
    'order' => 'name',
    'list_label' => 'Customer List',
    'ajax' => true,
    'search' => ['name', 'city'],
    'fields' => [
        'name' => ['label' => 'Name', 'type' => 'text', 'max' => 100],
        'email' => ['label' => 'Email', 'type' => 'email', 'max' => 140],
        'phone' => ['label' => 'Phone', 'type' => 'text', 'max' => 25],
        'city' => ['label' => 'City', 'type' => 'text', 'max' => 80],
    ],
];
```

PROGRAM / RUN_ME.md

```
# Customer Management with AJAX

PHP 8.0+ and the **pdo_mysql** extension, MySQL 8.0+/MariaDB.

1. Import `schema.sql` using phpMyAdmin or the `mysql` command line.
2. Configure `DB_HOST`, `DB_USER`, `DB_PASSWORD` environment variables (or edit defaults in `db.php` for
local development).
3. From this program folder run `php -S 127.0.0.1:8000`.
4. Visit http://127.0.0.1:8000/index.php and test create/edit/delete.
5. Try typing a search term into the live AJAX search field.

Do not deploy the default local root/blank-password database configuration to production.
This package has passed PHP syntax checks, not database-backed runtime tests.
```

PROGRAM / schema.sql

```
CREATE DATABASE IF NOT EXISTS `customer_db` CHARACTER SET utf8mb4;
USE `customer_db`;
CREATE TABLE IF NOT EXISTS `customers` (
  `customer_id` INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  `name` VARCHAR(100) NOT NULL,
  `email` VARCHAR(140) NOT NULL,
  `phone` VARCHAR(25) NOT NULL,
  `city` VARCHAR(80) NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

Common project files (reproduced in the appendix): common.php, db.php, delete.php, edit.php, index.php, rows.php, search.php, style.css.

EXPECTED OUTPUT

```
Insert -> new row appears in list.
Edit -> changes are persisted.
Delete -> record removed after confirmation.
Typing a search term -> matching rows update without full page reload.
```

Expected interaction only; no MySQL server or pdo_mysql driver available on the execution host.

Note: Run schema.sql in MySQL, set DB_HOST/DB_USER/DB_PASSWORD, then serve with PHP 8.0+ and the pdo_mysql extension. Search and CRUD use prepared statements; output is escaped; mutations use POST and CSRF checks.

05. Mobile Shop Management with AJAX

Aim: Implement mobile CRUD, AJAX search by brand or model using WHERE, and sort records by price using ORDER BY.

Logic: MySQL stores the records; PDO implements parameterized CRUD; JavaScript fetch updates search results without reloading the page.

PROGRAM / model.php

```
<?php
return [
    'title' => 'Mobile Shop Management with AJAX',
    'database' => 'mobile_db',
    'table' => 'mobiles',
    'pk' => 'mobile_id',
    'order' => 'price',
    'list_label' => 'Mobile Stock List',
    'ajax' => true,
    'search' => ['brand', 'model'],
    'fields' => [
        'brand' => ['label' => 'Brand', 'type' => 'text', 'max' => 80],
        'model' => ['label' => 'Model', 'type' => 'text', 'max' => 120],
        'price' => ['label' => 'Price', 'type' => 'number', 'max' => 30],
        'stock' => ['label' => 'Stock', 'type' => 'integer', 'max' => 20],
    ],
];
```

PROGRAM / RUN_ME.md

```
# Mobile Shop Management with AJAX

PHP 8.0+ and the **pdo_mysql** extension, MySQL 8.0+/MariaDB.

1. Import `schema.sql` using phpMyAdmin or the `mysql` command line.
2. Configure `DB_HOST`, `DB_USER`, `DB_PASSWORD` environment variables (or edit defaults in `db.php` for
local development).
3. From this program folder run `php -S 127.0.0.1:8000`.
4. Visit http://127.0.0.1:8000/index.php and test create/edit/delete.
5. Try typing a search term into the live AJAX search field.

Do not deploy the default local root/blank-password database configuration to production.
This package has passed PHP syntax checks, not database-backed runtime tests.
```

PROGRAM / schema.sql

```
CREATE DATABASE IF NOT EXISTS `mobile_db` CHARACTER SET utf8mb4;
USE `mobile_db`;
CREATE TABLE IF NOT EXISTS `mobiles` (
  `mobile_id` INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  `brand` VARCHAR(80) NOT NULL,
  `model` VARCHAR(120) NOT NULL,
  `price` DECIMAL(12,2) NOT NULL CHECK (`price` >= 0),
  `stock` INT UNSIGNED NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

Common project files (reproduced in the appendix): common.php, db.php, delete.php, edit.php, index.php, rows.php, search.php, style.css.

EXPECTED OUTPUT

```
Insert -> new row appears in list.
Edit -> changes are persisted.
Delete -> record removed after confirmation.
Typing a search term -> matching rows update without full page reload.
```

Expected interaction only; no MySQL server or pdo_mysql driver available on the execution host.

Note: Run schema.sql in MySQL, set DB_HOST/DB_USER/DB_PASSWORD, then serve with PHP 8.0+ and the pdo_mysql extension. Search and CRUD use prepared statements; output is escaped; mutations use POST and CSRF checks. Index requires sorting by price, whereas supplied reference code sorted by brand; the index takes priority.

APPENDIX

Shared Project Files

The following files are required by multiple programs. Each is printed in full here once; the respective practicals list the files they use.

COMMON FILE / common.php

Used by: Assignment 2: 01, Assignment 2: 02, Assignment 2: 03, Assignment 2: 04, Assignment 2: 05

```
<?php
declare(strict_types=1);

session_start();
$model = require __DIR__ . '/model.php';
require_once __DIR__ . '/db.php';

function e($value): string {
    return htmlspecialchars((string)$value, ENT_QUOTES | ENT_SUBSTITUTE, 'UTF-8');
}
function token(): string {
    if (empty($_SESSION['csrf'])) {
        $_SESSION['csrf'] = bin2hex(random_bytes(32));
    }
    return $_SESSION['csrf'];
}
function check_token(): void {
    if (!hash_equals(token(), (string)($_POST['csrf'] ?? ''))) {
        http_response_code(403); exit('Invalid CSRF token.');
    }
}
function positive_id($input): int {
    $id = filter_var($input, FILTER_VALIDATE_INT);
    if ($id === false || $id < 1) {
        http_response_code(400); exit('Invalid record identifier.');
    }
    return $id;
}
function validate_input(array $model): array {
    $values = [];
    foreach ($model['fields'] as $key => $definition) {
        $value = trim((string)($_POST[$key] ?? ''));
        if ($value === '') {
            throw new InvalidArgumentException($definition['label'] . ' is required.');
        }
        switch ($definition['type']) {
            case 'email':
                if (!filter_var($value, FILTER_VALIDATE_EMAIL)) {
                    throw new InvalidArgumentException('Invalid email address.');
                }
                break;
            case 'number':
                if (!is_numeric($value) || (float)$value < 0) {
                    throw new InvalidArgumentException('Invalid amount.');
                }
                break;
            case 'integer':
                if (!ctype_digit($value)) {
                    throw new InvalidArgumentException('Enter a non-negative integer.');
                }
                break;
        }
        if (strlen($value) > ($definition['max'] ?? 1000)) {
            throw new InvalidArgumentException($definition['label'] . ' is too long.');
        }
        $values[$key] = $value;
    }
    return $values;
}
function fetch_rows(array $model, string $search = ''): array {
    $table = $model['table'];
    $order = $model['order'];
```

```
    $sql = "SELECT * FROM `$table`";
    $params = [];
    if ($search !== '' && !empty($model['search'])) {
        $clauses = [];
        foreach ($model['search'] as $i => $column) {
            $placeholder = ":q$i";
            $clauses[] = "`$column` LIKE $placeholder";
            $params[$placeholder] = '%' . $search . '%';
        }
        $sql .= ' WHERE ' . implode(' OR ', $clauses);
    }
    $sql .= " ORDER BY `$order` ASC";
    $stmt = db($model)->prepare($sql);
    $stmt->execute($params);
    return $stmt->fetchAll();
}
function goto_home(): void {
    header('Location: index.php');
    exit;
}
```

COMMON FILE / db.php

Used by: Assignment 2: 01, Assignment 2: 02, Assignment 2: 03, Assignment 2: 04, Assignment 2: 05

```
<?php
declare(strict_types=1);

function db(array $model): PDO {
    static $db = null;
    if ($db instanceof PDO) return $db;
    $host = getenv('DB_HOST') ?: '127.0.0.1';
    $user = getenv('DB_USER') ?: 'root';
    $pass = getenv('DB_PASSWORD') ?: '';
    $database = $model['database'];
    $dsn = "mysql:host=$host;dbname=$database;charset=utf8mb4";
    $db = new PDO($dsn, $user, $pass, [
        PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
        PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
        PDO::ATTR_EMULATE_PREPARES => false,
    ]);
    return $db;
}
```

COMMON FILE / delete.php

Used by: Assignment 2: 01, Assignment 2: 02, Assignment 2: 03, Assignment 2: 04, Assignment 2: 05

```
<?php
require_once __DIR__ . '/common.php';
if ($_SERVER['REQUEST_METHOD'] !== 'POST') {
    http_response_code(405); exit('POST required.');
}
check_token();
$id = positive_id($_POST['id'] ?? null);
$table = $model['table'];
$pk = $model['pk'];
$stmt = db($model)->prepare("DELETE FROM `$table` WHERE `$pk` = :id");
$stmt->execute(['id' => $id]);
goto_home();
```

COMMON FILE / edit.php

Used by: Assignment 2: 01, Assignment 2: 02, Assignment 2: 03, Assignment 2: 04, Assignment 2: 05

```
<?php
require_once __DIR__ . '/common.php';
$id = positive_id($_GET['id'] ?? null);
$table = $model['table'];
$pk = $model['pk'];
$stmt = db($model)->prepare("SELECT * FROM `$table` WHERE `$pk` = :id");
$stmt->execute([':id' => $id]);
$row = $stmt->fetch();
if (!$row) { http_response_code(404); exit('Record not found.'); }
$error = '';
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
```

```
    check_token();
    try {
        $values = validate_input($model);
        $set = implode(', ', array_map(fn($k) => "`$k` = :$k", array_keys($values)));
        $stmt = db($model)->prepare("UPDATE `$table` SET $set WHERE `$pk` = :id");
        $stmt->execute($values + ['id' => $id]);
        goto_home();
    } catch (InvalidArgumentException $ex) {
        $error = $ex->getMessage();
        $row = array_replace($row, $_POST);
    }
}
?>
<!doctype html><html lang="en"><head><meta charset="utf-8">
<meta name="viewport" content="width=device-width,initial-scale=1">
<title>Edit <?= e($model['title']) ?></title>
<link rel="stylesheet" href="style.css"></head><body>
<header><h1>Edit <?= e($model['title']) ?></h1><p>Record #<?= $id ?></p></header>
<main><section class="card"><h2>Edit record</h2>
<?php if ($error): ?><p class="alert" role="alert"><?= e($error) ?></p><?php endif; ?>
<form method="post"><input type="hidden" name="csrf" value="<?= e(token()) ?>">
<div class="fields">
<?php foreach ($model['fields'] as $key => $def): ?>
<div><label for="f_<?= e($key) ?>"><?= e($def['label']) ?></label>
<?php if ($def['type'] === 'textarea'): ?>
<textarea required id="f_<?= e($key) ?>" name="<?= e($key) ?>"><?= e($row[$key]) ?></textarea>
<?php else: ?>
<input required id="f_<?= e($key) ?>" name="<?= e($key) ?>"
 value="<?= e($row[$key]) ?>"
 type="<?= e($def['type'] === 'integer' ? 'number' :
   ($def['type'] === 'number' ? 'number' : $def['type'])) ?>"
 <?= in_array($def['type'], ['number','integer'], true)
      ? 'min="0" step="' . ($def['type'] === 'integer' ? '1' : '0.01') . '"' : '' ?>>
<?php endif; ?></div>
<?php endforeach; ?></div><div class="actions">
<button type="submit">Update record</button>
<a href="index.php" class="button muted">Cancel</a></div></form>
</section></main></body></html>
```

COMMON FILE / index.php

Used by: Assignment 2: 01, Assignment 2: 02, Assignment 2: 03, Assignment 2: 04, Assignment 2: 05

```
<?php
require_once __DIR__ . '/common.php';
require_once __DIR__ . '/rows.php';
$error = '';
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    check_token();
    try {
        $values = validate_input($model);
        $names = array_keys($values);
        $columns = implode(', ', array_map(fn($n) => "`$n`", $names));
        $holders = implode(', ', array_map(fn($n) => ":$n", $names));
        $table = $model['table'];
        $stmt = db($model)->prepare(
            "INSERT INTO `$table` ($columns) VALUES ($holders)"
        );
        $stmt->execute($values);
        goto_home();
    } catch (InvalidArgumentException $ex) {
        $error = $ex->getMessage();
    }
}
$search = trim((string)($_GET['q'] ?? ''));
try {
    $records = fetch_rows($model, $search);
} catch (PDOException $ex) {
    http_response_code(500);
    $error = 'Database connection failed. Check setup and credentials.';
    $records = [];
}
?>
<!doctype html><html lang="en"><head><meta charset="utf-8">
<meta name="viewport" content="width=device-width,initial-scale=1">
<title><?= e($model['title']) ?></title>
```

```
<link rel="stylesheet" href="style.css"></head><body>
<header><h1><?= e($model['title']) ?></h1>
<p>PHP · MySQL · structured records</p></header>
<main>
<section class="card"><h2>Add new record</h2>
<?php if ($error): ?><p role="alert" class="alert"><?= e($error) ?></p><?php endif; ?>
<form method="post" action="index.php"><input type="hidden" name="csrf"
 value="<?= e(token()) ?>"><div class="fields">
<?php foreach ($model['fields'] as $key => $def): ?>
<div><label for="f_<?= e($key) ?>"><?= e($def['label']) ?></label>
<?php if ($def['type'] === 'textarea'): ?>
<textarea required id="f_<?= e($key) ?>" name="<?= e($key) ?>"></textarea>
<?php else: ?>
<input required id="f_<?= e($key) ?>" name="<?= e($key) ?>"
 type="<?= e($def['type'] === 'integer' ? 'number' :
   ($def['type'] === 'number' ? 'number' : $def['type'])) ?>"
 <?= in_array($def['type'], ['number','integer'], true)
      ? 'min="0" step="' . ($def['type'] === 'integer' ? '1' : '0.01') . '"' : '' ?>>
<?php endif; ?></div>
<?php endforeach; ?></div><div class="actions">
<button type="submit">Save record</button></div></form>
</section>
<section class="card"><h2><?= e($model['list_label']) ?></h2>
<?php if ($model['search']): ?>
<?php if ($model['ajax']): ?>
<div class="search"><label for="live-search">Live search</label>
<input id="live-search" autocomplete="off" placeholder="Search records" aria-label="Live search"></div>
<?php else: ?>
<form class="search" method="get"><label for="search">Search product / category</label>
<input id="search" name="q" value="<?= e($search) ?>">
<div class="actions"><button type="submit">Search</button>
<a class="button muted" href="index.php">Clear</a></div></form>
<?php endif; endif; ?>
<div class="table-wrap"><table><thead><tr><th>ID</th>
<?php foreach ($model['fields'] as $def): ?><th><?= e($def['label']) ?></th><?php endforeach; ?>
<th>Actions</th></tr></thead><tbody id="data-body">
<?php render_rows($records, $model); ?>
</tbody></table></div></section></main>
<?php if ($model['ajax']): ?>
<script>
const searchBox = document.querySelector('#live-search');
const tbody = document.querySelector('#data-body');
let waitHandle;
searchBox.addEventListener('input', () => {
  clearTimeout(waitHandle);
  waitHandle = setTimeout(async () => {
    try {
      const response = await fetch('search.php?q=' +
        encodeURIComponent(searchBox.value));
      if (!response.ok) throw new Error('Search failed');
      tbody.innerHTML = await response.text();
    } catch (_) {
      tbody.innerHTML = '<tr><td>Unable to load results.</td></tr>';
    }
  }, 180);
});
</script>
<?php endif; ?>
</body></html>
```

COMMON FILE / rows.php

Used by: Assignment 2: 01, Assignment 2: 02, Assignment 2: 03, Assignment 2: 04, Assignment 2: 05

```
<?php
function render_rows(array $records, array $model): void {
    if (!$records) {
        echo '<tr><td colspan="' . (count($model['fields']) + 2) . '">'
            . 'No matching records.</td></tr>';
        return;
    }
    foreach ($records as $row) {
        echo '<tr><td>' . e($row[$model['pk']]) . '</td>';
        foreach (array_keys($model['fields']) as $name) {
            echo '<td>' . e($row[$name]) . '</td>';
        }
```

```
        $id = (int)$row[$model['pk']];
        echo '<td><a href="edit.php?id=' . $id . '">Edit</a> ';
        echo '<form class="inline" method="post" action="delete.php"'
             . ' onsubmit="return confirm(&quot;Delete this record?&quot;)">';
        echo '<input type="hidden" name="id" value="' . $id . '">';
        echo '<input type="hidden" name="csrf" value="' . e(token()) . '">';
        echo '<button type="submit">Delete</button></form></td></tr>';
    }
}
```

COMMON FILE / search.php

Used by: Assignment 2: 04, Assignment 2: 05

```
<?php
require_once __DIR__ . '/common.php';
require_once __DIR__ . '/rows.php';
header('Content-Type: text/html; charset=utf-8');
try {
    $q = trim((string)($_GET['q'] ?? ''));
    render_rows(fetch_rows($model, $q), $model);
} catch (PDOException $ex) {
    http_response_code(500);
    echo '<tr><td>Search unavailable.</td></tr>';
}
```

COMMON FILE / style.css

Used by: Assignment 2: 01, Assignment 2: 02, Assignment 2: 03, Assignment 2: 04, Assignment 2: 05

```
* { box-sizing:border-box; }
body { margin:0; font:15px/1.6 system-ui,-apple-system,Segoe UI,sans-serif;
  color:#17344a; background:#f3f7f9; }
header { background:#123047; color:white; padding:25px max(24px,calc((100vw - 1040px)/2)); }
header h1 { margin:0 0 3px; font-size:27px; }
header p { margin:0; color:#c6d9e2; }
main { max-width:1040px; margin:26px auto; padding:0 18px 35px; }
.card { background:white; border:1px solid #e1e8ed; border-radius:10px;
  padding:22px; margin-bottom:22px; box-shadow:0 2px 10px #1230470b; }
h2 { margin:0 0 20px; font-size:19px; }
.fields { display:grid; grid-template-columns:repeat(2,minmax(0,1fr)); gap:15px; }
.fields label { display:block; font-weight:650; font-size:13px; margin-bottom:5px; }
input,textarea { display:block; width:100%; padding:11px; border-radius:6px;
  border:1px solid #b9c8d0; background:#fff; font:inherit; }
input:focus,textarea:focus,button:focus,a:focus-visible {
  outline:2px solid #137d8c; outline-offset:2px; }
textarea { min-height:90px; resize:vertical; }
button,.button { display:inline-block; background:#0a6774; color:white;
  padding:9px 15px; border:0; border-radius:6px; font:inherit;
  font-weight:600; cursor:pointer; text-decoration:none; }
.button.muted { color:#17344a; background:#e6edf0; }
.actions { display:flex; gap:12px; align-items:center; flex-wrap:wrap; margin-top:19px; }
.alert { color:#933; background:#fff2f0; border-left:3px solid #c43c3c;
  padding:10px 14px; margin-bottom:15px; }
.table-wrap { overflow-x:auto; }
table { width:100%; border-collapse:collapse; }
th { background:#e4f0f3; color:#17344a; text-align:left; }
th,td { padding:11px; border-bottom:1px solid #dce6e9; vertical-align:top; }
.inline { display:inline; }
.inline button { background:#fff4f2; color:#9c2318; padding:5px 9px; }
a { color:#0a6774; font-weight:600; }
.search { max-width:450px; margin-bottom:16px; }
small { color:#627785; }
@media(max-width:680px){ .fields {grid-template-columns:1fr;} }
```